

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikorkontroler

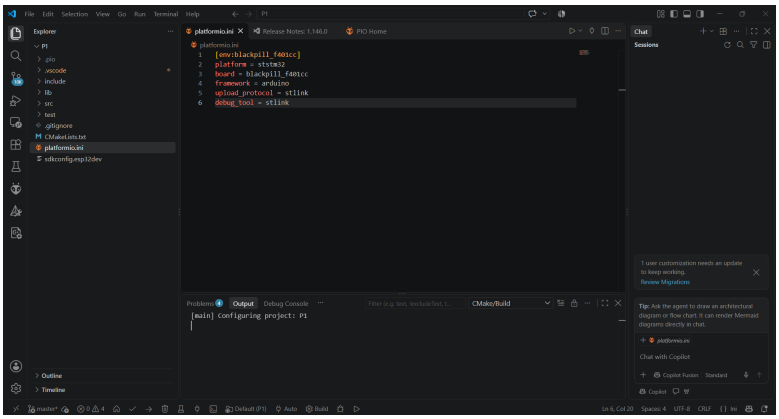
Danendra Nusi Putra Rahanda - 5024241052

2026

1 Langkah-Langkah Percobaan

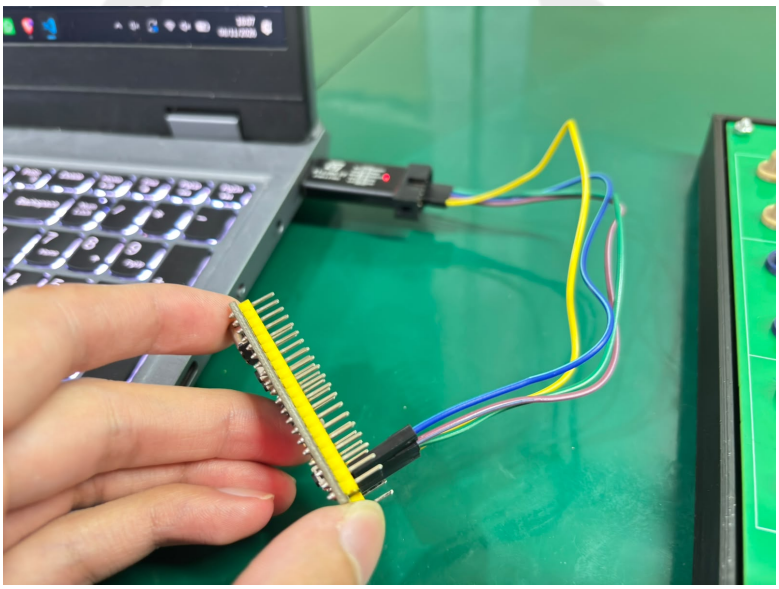
1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Buat project PlatformIO Modul 1, lalu isi `platformio.ini` dengan board `blackpill_f401cc`, framework `arduino`, dan `upload_protocol = stlink`. Tunggu proses refresh selesai.



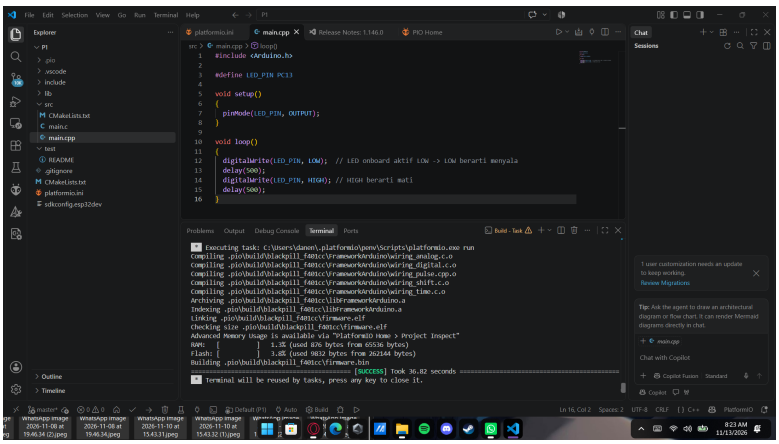
Gambar 1: Konfigurasi `platformio.ini` Percobaan 1

2. Hubungkan pin SWDIO, SWCLK, GND, dan 3V3 ST-Link ke Blackpill, lalu colokkan ST-Link ke komputer melalui USB.



Gambar 2: Rangkaian ST-Link ke STM32 Blackpill

3. Tulis kode Blink pada `src/main.cpp`, lalu klik *Build* dan *Upload*.



Gambar 3: Hasil *Build* dan *Upload* Percobaan 1

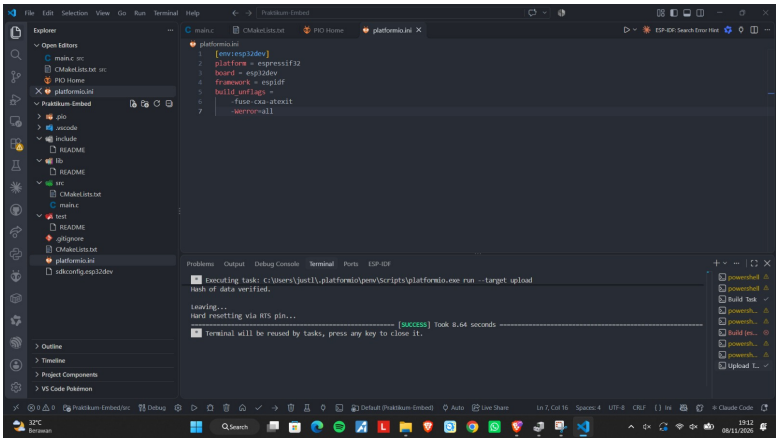
4. Amati LED onboard PC13 yang menyala 500 ms dan mati 500 ms secara berulang.



Gambar 4: LED onboard PC13 berkedip

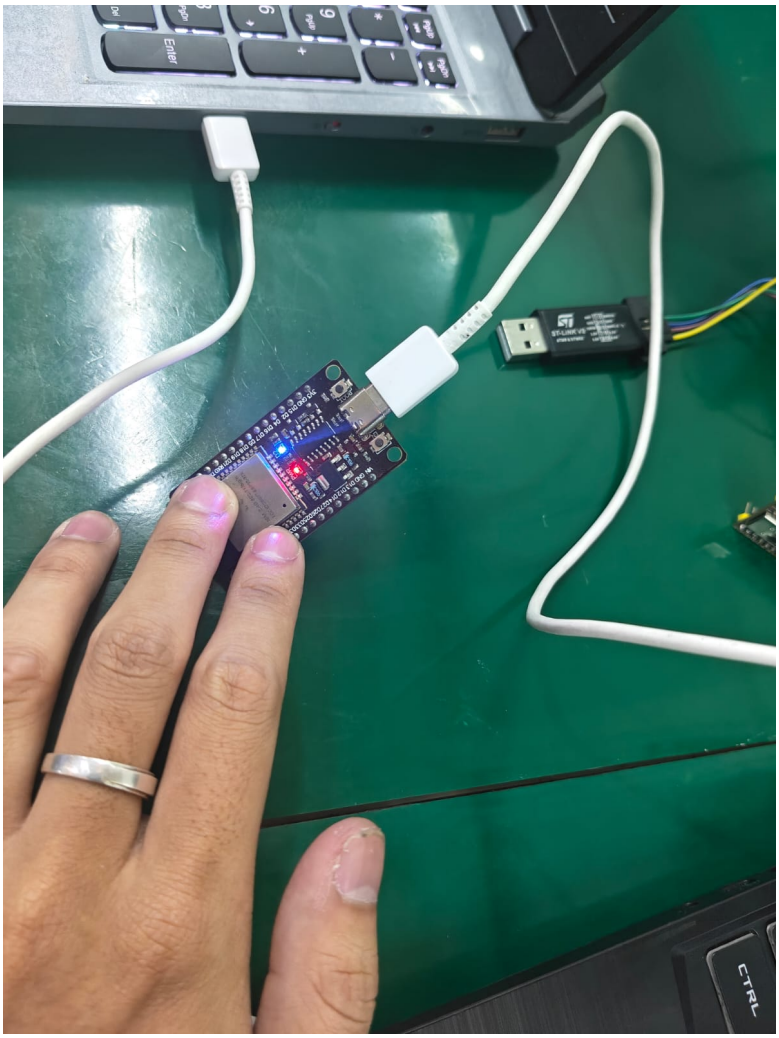
1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Ganti isi `platformio.ini` dengan board `esp32dev` dan framework `espidf`, lalu tunggu refresh selesai.



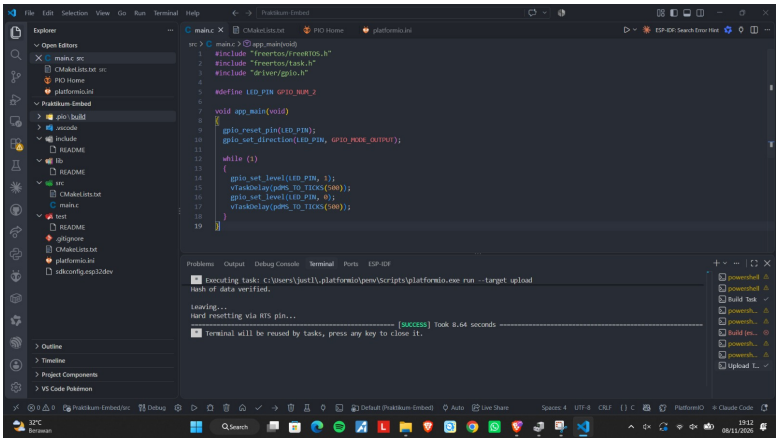
Gambar 5: Konfigurasi `platformio.ini` Percobaan 2

2. Colokkan ESP32 ke komputer melalui USB dan pastikan port terdeteksi.



Gambar 6: Port ESP32 terdeteksi

3. Hapus `src/main.cpp`, buat `src/main.c` berisi kode Blink ESP-IDF, lalu klik *Build* dan *Upload*.

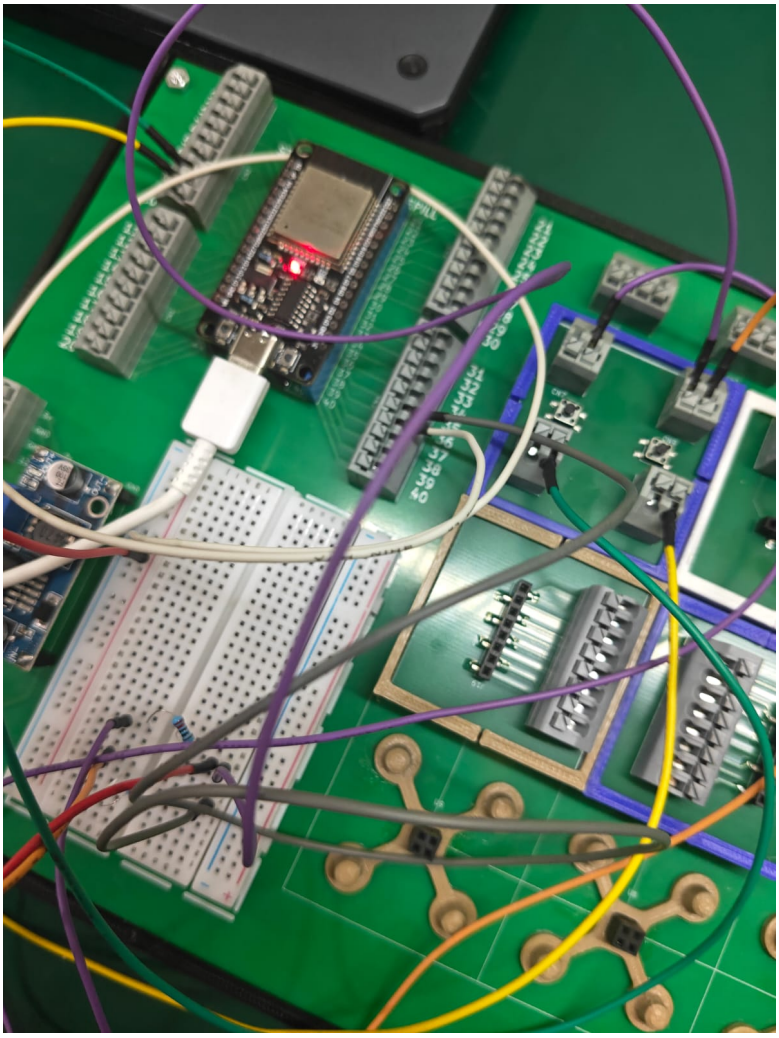


Gambar 7: Hasil *Build* dan *Upload* Percobaan 2

4. Amati LED onboard GPIO 2 yang berkedip, lalu bandingkan pemrograman ESP-IDF dengan Arduino.

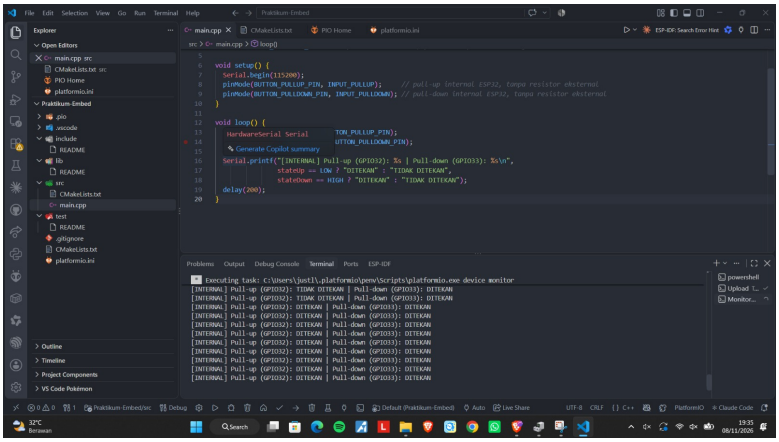
1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal vs Internal

1. Ganti `platformio.ini` kembali ke framework `arduino`, hapus `src/main.c`, lalu buat `src/main.cpp`.
2. Rangkai dua tombol: GPIO 32 dengan *pull-up* eksternal 10 kΩ dan GPIO 33 dengan *pull-down* eksternal 10 kΩ.



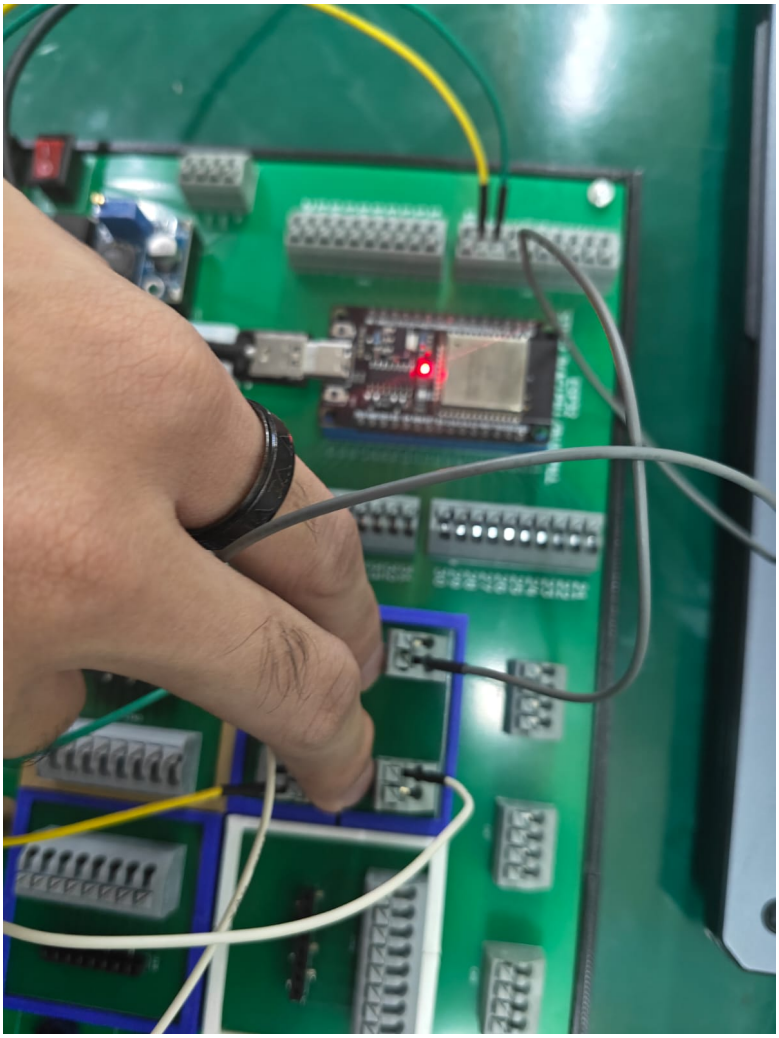
Gambar 8: Rangkaian *pull-up* dan *pull-down* eksternal

3. Upload Kode A (eksternal), tekan kedua tombol, dan catat hasilnya di Serial Monitor.



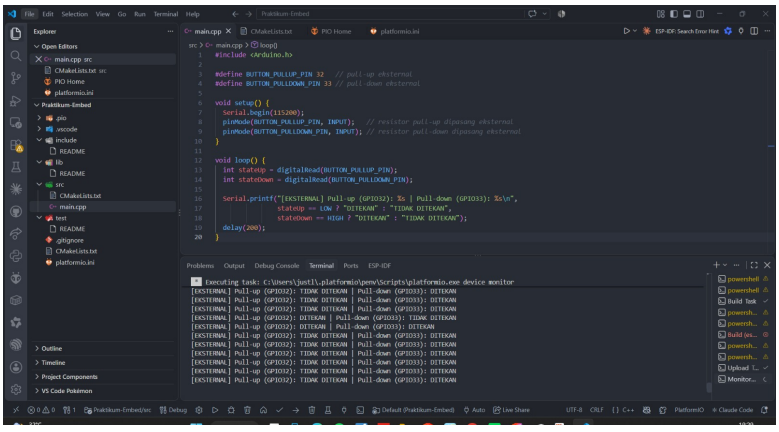
Gambar 9: Serial Monitor hasil Kode A (eksternal)

4. Lepas kedua resistor eksternal sehingga tombol tetap terpasang tanpa resistor.



Gambar 10: Rangkaian tanpa resistor eksternal

5. Upload Kode B (internal), tekan kedua tombol, dan catat hasilnya di Serial Monitor.

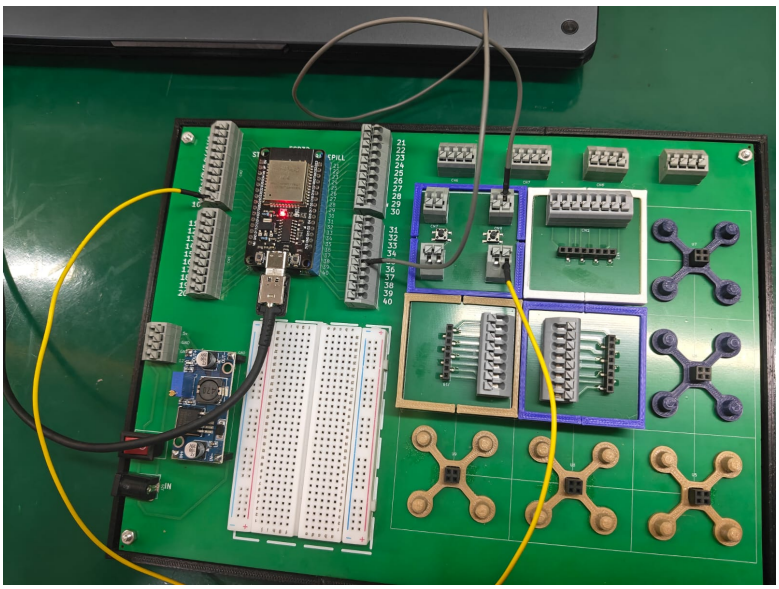


Gambar 11: Serial Monitor hasil Kode B (internal)

6. Bandingkan hasil Kode A dan Kode B.

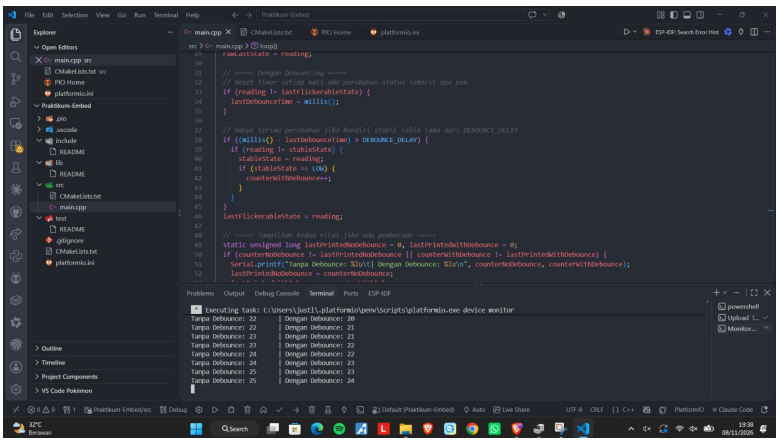
1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Pasang tombol pada GPIO 25 dan GND, dengan konfigurasi `INPUT_PULLUP`.



Gambar 12: Rangkaian tombol pada GPIO 25

2. Upload kode dengan dua penghitung (tanpa dan dengan *debounce*), lalu buka Serial Monitor.
3. Tekan tombol satu kali, secara cepat dan lambat, lalu bandingkan kedua penghitung.

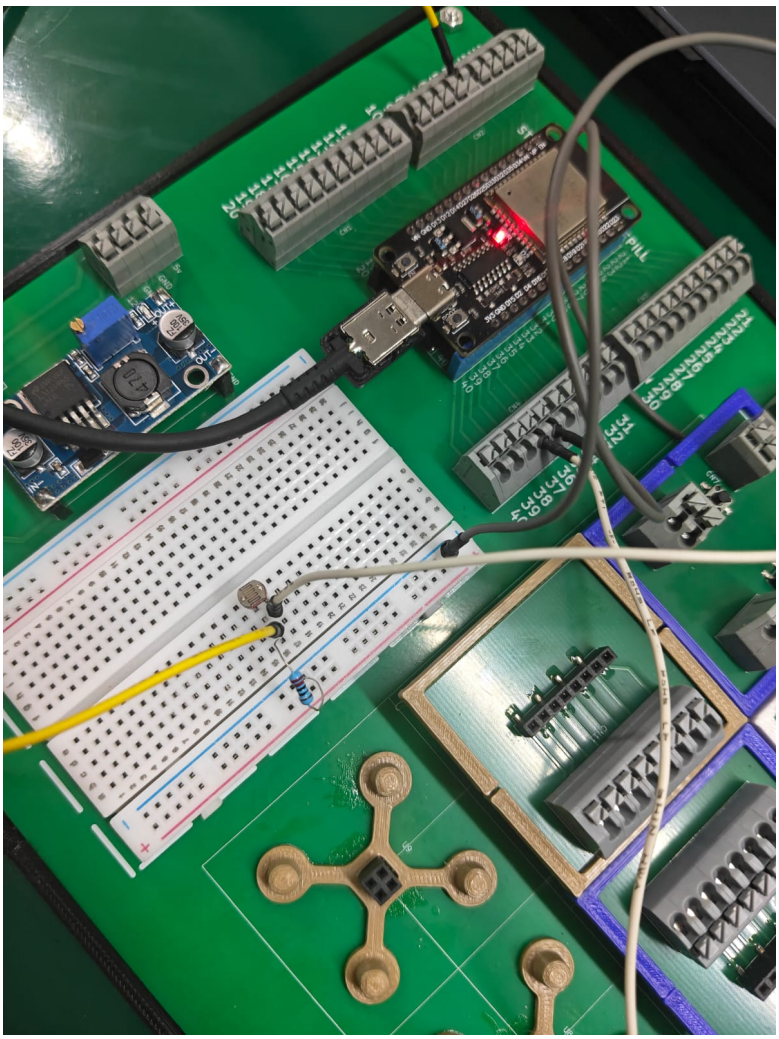


Gambar 13: Serial Monitor perbandingan penghitung tanpa dan dengan *debounce*

4. Ubah `DEBOUNCE_DELAY` menjadi 10 ms, upload, lalu tekan tombol dan amati hasilnya.
5. Ubah `DEBOUNCE_DELAY` menjadi 200 ms, upload, lalu tekan tombol berulang kali dengan cepat.
6. Bandingkan hasil untuk nilai 10 ms, 50 ms, dan 200 ms.

1.5 Percobaan 5: Pembacaan ADC dengan LDR

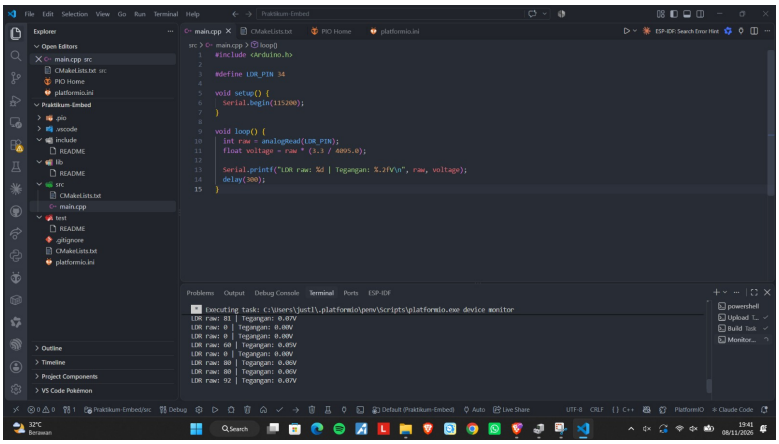
1. Rangkai LDR dan resistor 10 kΩ sebagai pembagi tegangan, lalu hubungkan titik tengahnya ke GPIO 34.



Gambar 14: Rangkaian pembagi tegangan LDR

2. Tulis kode pembacaan ADC, klik *Build* dan *Upload*, lalu buka Serial Monitor.

3. Tutup LDR dengan tangan, lalu sinari dengan lampu, dan catat nilai mentah serta tegangan yang terbaca.



Gambar 15: Serial Monitor nilai ADC pada kondisi gelap dan terang

2 Analisis Hasil Percobaan

Pada Percobaan 1, LED onboard STM32 Blackpill di pin PC13 menyala selama 500 ms dan mati selama 500 ms secara berulang sesuai nilai `delay(500)` pada kode, dan LED tersebut bersifat aktif LOW sehingga menyala saat pin diberi LOW. Pada Percobaan 2, LED onboard ESP32 di GPIO 2 berkedip dengan pola yang sama menggunakan ESP-IDF. Perbedaan keduanya terletak pada struktur program: STM32duino menyederhanakan pemrograman lewat fungsi Arduino yang berjalan di atas HAL, sedangkan ESP-IDF memakai `app_main()` yang berjalan sebagai task FreeRTOS dengan jeda `vTaskDelay()` yang tidak menahan task lain. Kode ESP-IDF lebih panjang dan proses *build*-nya lebih berat, tetapi aksesnya ke fitur RTOS lebih langsung.

Pada kedua percobaan tersebut, proses *build* sempat tidak merespon dan menghasilkan *error* pada ketiga laptop kelompok kami, dengan pesan yang kurang kami pahami maksudnya sehingga kami meminta bantuan dari asisten yang ada di dalam ruangan praktikum, dan baru berhasil setelah troubleshooting berupa Install ulang framework dan create ulang project di PlatformIO (PIO). Kemungkinan penyebabnya adalah *build* pertama PlatformIO yang harus mengunduh platform dan *toolchain* berukuran besar, *build* yang dijalankan sebelum proses refresh `platformio.ini` selesai, atau masalah path dan sisa file saat berpindah framework. Karena gejalanya sama pada ketiga laptop dan kode tidak berubah, penyebabnya lebih bersifat lingkungan pengembangan daripada kesalahan kode.

Pada Percobaan 3, pembacaan tombol dengan resistor eksternal maupun internal memberikan hasil yang sama dan sesuai teori. Tombol *pull-up* (GPIO 32) terbaca HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan tombol *pull-down* (GPIO 33) terbaca LOW saat tidak ditekan dan HIGH saat ditekan. Resistor *pull* diperlukan agar pin tidak *floating* sehingga levelnya pasti saat tombol terbuka. Hasil eksternal dan internal sama karena fungsinya sama, tetapi resistor internal ESP32 bernilai sekitar 45 kΩ, lebih lemah daripada 10 kΩ eksternal, sehingga lebih mudah terpengaruh *noise*. Selain itu, pin input-only GPIO 34 sampai 39 tidak memiliki resistor *pull* internal.

Pada Percobaan 4, `counterWithDebounce` selalu bertambah tepat satu untuk setiap tekanan, sedangkan `counterNoDebounce` pada beberapa tekanan bertambah lebih dari satu. Hal ini disebabkan kontak mekanik tombol yang memantul beberapa kali sebelum menetap, sementara loop program yang cepat membaca setiap pantulan sebagai transisi tersendiri. Dengan `DEBOUNCE_DELAY` 10 ms, 50 ms, dan 200 ms, nilai delay yang kecil berisiko meloloskan getaran, sedangkan nilai yang besar membuat tombol terasa lambat dan tekanan cepat tidak terhitung, sehingga 50 ms menjadi kompromi yang baik.

Pada Percobaan 5, nilai ADC berubah mengikuti intensitas cahaya sesuai prinsip pembagi tegangan, dengan tegangan keluaran

$$V_o = 3{,}3 \times \frac{10\,\mathrm{k}\Omega}{R_{LDR} + 10\,\mathrm{k}\Omega}, \tag{1}$$

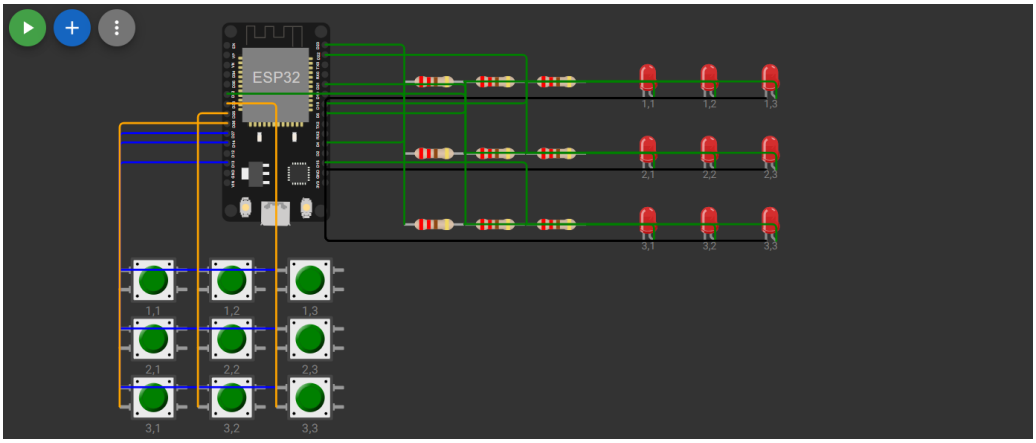
sehingga saat terang hambatan LDR turun dan V_o naik, sedangkan saat gelap hambatannya naik dan V_o turun. ADC 12-bit memiliki 4096 level sehingga sinyal analog dapat merepresentasikan banyak kondisi, berbeda dari input digital yang hanya mengenal HIGH atau LOW. Pembacaan ADC ESP32 tidak sepenuhnya linear dan sedikit berfluktuasi akibat *noise*, sehingga konversi ke tegangan hanya merupakan perkiraan.

Berdasarkan seluruh hasil tersebut, percobaan dapat dinilai berhasil karena keluaran setiap percobaan sesuai dengan teori, dan satu-satunya hambatan berasal dari lingkungan pengembangan pada tahap *build*, bukan dari kode maupun perangkaian.

3 Tugas Modul

3.1 Rangkaian dan Cara Kerja

Sembilan pushbutton disusun sebagai matriks 3×3 dan sembilan LED disusun sebagai matriks 3×3 dengan resistor 220 Ω pada masing-masing LED. Baris tombol dihubungkan ke GPIO 13, 14, dan 27 sebagai output yang dipindai satu per satu, sedangkan kolom tombol dihubungkan ke GPIO 26, 25, dan 33 dengan `INPUT_PULLUP`. Tombol pada baris r kolom c menukar status LED pada baris r kolom c setiap kali tekanan yang stabil terdeteksi, dengan debouncing 50 ms pada setiap tombol seperti pada Percobaan 4.



Gambar 16: Rangkaian matriks pushbutton 3x3 dan matriks LED 3x3 pada Wokwi

3.2 Source Code

```
const uint8_t ROW_PINS[3] = {13, 14, 27};
const uint8_t COL_PINS[3] = {26, 25, 33};
const uint8_t LED_PINS[3][3] = {
  {4, 5, 18},
  {19, 21, 22},
  {23, 32, 15}
};

const unsigned long DEBOUNCE_DELAY = 50; // ms

bool ledOn[3][3];
int lastReading[3][3];
int stableState[3][3];
unsigned long lastChangeTime[3][3];

void setup() {
  Serial.begin(115200);
  for (int c = 0; c < 3; c++) {
    pinMode(COL_PINS[c], INPUT_PULLUP);
  }
  for (int r = 0; r < 3; r++) {
    pinMode(ROW_PINS[r], INPUT);
    for (int c = 0; c < 3; c++) {
```

```
      pinMode(LED_PINS[r][c], OUTPUT);
      digitalWrite(LED_PINS[r][c], LOW);
      ledOn[r][c] = false;
      lastReading[r][c] = HIGH;
      stableState[r][c] = HIGH;
      lastChangeTime[r][c] = 0;
    }
  }
}

void loop() {
  for (int r = 0; r < 3; r++) {
    pinMode(ROW_PINS[r], OUTPUT);
    digitalWrite(ROW_PINS[r], LOW);
    delayMicroseconds(50);

    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(COL_PINS[c]);

      if (reading != lastReading[r][c]) {
        lastChangeTime[r][c] = millis();
      }

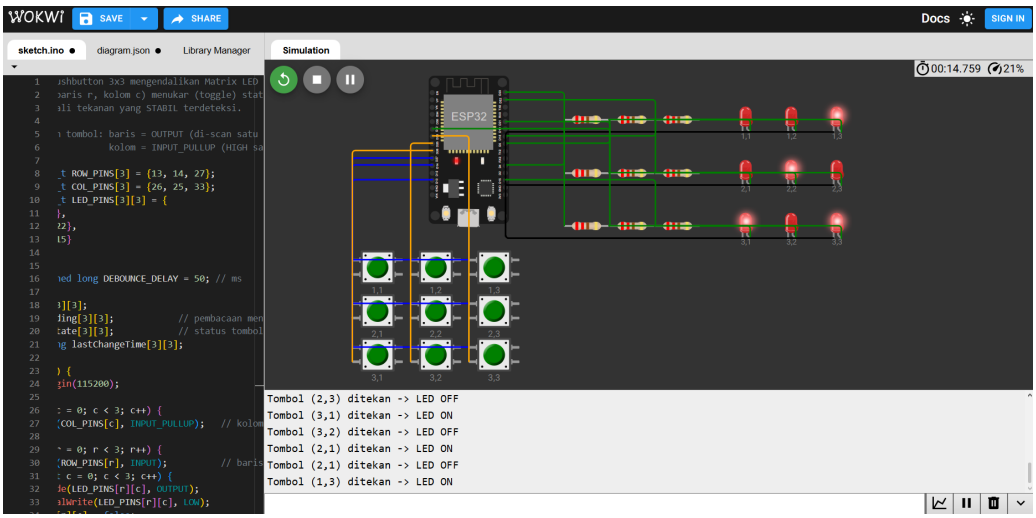
      if ((millis() - lastChangeTime[r][c]) > DEBOUNCE_DELAY &&
          reading != stableState[r][c]) {
        stableState[r][c] = reading;

        if (stableState[r][c] == LOW) {
          ledOn[r][c] = !ledOn[r][c];
          digitalWrite(LED_PINS[r][c], ledOn[r][c] ? HIGH : LOW);
          Serial.printf("Tombol (%d,%d) ditekan -> LED %s\n",
                        r + 1, c + 1, ledOn[r][c] ? "ON" : "OFF");
        }
      }
      lastReading[r][c] = reading;
    }

    pinMode(ROW_PINS[r], INPUT);
  }
}
```

3.3 Hasil Simulasi

Saat tombol ditekan, LED pada posisi yang sama berganti status (menyala atau mati), dan setiap tekanan tercatat satu kali pada Serial Monitor.



Gambar 17: Hasil simulasi dengan beberapa LED menyala setelah tombol ditekan

```
Tombol (1,1) ditekan -> LED ON
Tombol (1,2) ditekan -> LED ON
Tombol (1,3) ditekan -> LED ON
Tombol (2,1) ditekan -> LED ON
Tombol (3,1) ditekan -> LED ON
Tombol (3,1) ditekan -> LED OFF
Tombol (3,1) ditekan -> LED ON
Tombol (2,2) ditekan -> LED ON
Tombol (2,3) ditekan -> LED ON
Tombol (3,2) ditekan -> LED ON
Tombol (3,3) ditekan -> LED ON
Tombol (2,1) ditekan -> LED OFF
Tombol (1,1) ditekan -> LED OFF
Tombol (3,1) ditekan -> LED OFF
Tombol (1,2) ditekan -> LED OFF
Tombol (1,3) ditekan -> LED OFF
Tombol (2,3) ditekan -> LED OFF
Tombol (3,1) ditekan -> LED ON
Tombol (3,2) ditekan -> LED OFF
Tombol (2,1) ditekan -> LED ON
Tombol (2,1) ditekan -> LED OFF
Tombol (1,3) ditekan -> LED ON
```

Gambar 18: Serial Monitor yang mencatat tekanan tombol

3.4 Jawaban Pertanyaan Analisis

1. Debouncing tetap diperlukan karena bouncing berasal dari sifat mekanis kontak tombol, bukan dari jumlah tombol yang ditekan. Saat kontak bersentuhan atau terlepas, kontak memantul beberapa kali sehingga sinyal bergantian HIGH dan LOW sesaat sebelum stabil. Tanpa debouncing, satu tekanan dapat membuat LED berganti status berkali-kali, sedangkan dengan debouncing status baru hanya diterima setelah sinyal stabil lebih dari 50 ms.

2. Kolom tombol memakai *pull-up* internal (`INPUT_PULLUP`), sehingga pin bernilai HIGH saat tombol tidak ditekan dan LOW saat tombol ditekan karena terhubung ke baris yang sedang bernilai LOW. Kondisi tidak ditekan tidak boleh dibiarkan *floating* karena pin yang tidak terhubung ke level tegangan apa pun akan menangkap *noise* sehingga nilainya acak, dan LED dapat berubah sendiri tanpa ada tombol yang ditekan.

3. Digital input hanya membedakan dua kondisi, HIGH atau LOW, berdasarkan ambang tegangan, sehingga tombol hanya dapat menyatakan ditekan atau tidak. Analog input melalui ADC mengubah tegangan yang kontinu menjadi angka, misalnya 0 sampai 4095 pada ADC 12-bit ESP32. Karena hambatan LDR berubah halus mengikuti cahaya, tegangan pada pin juga ber-

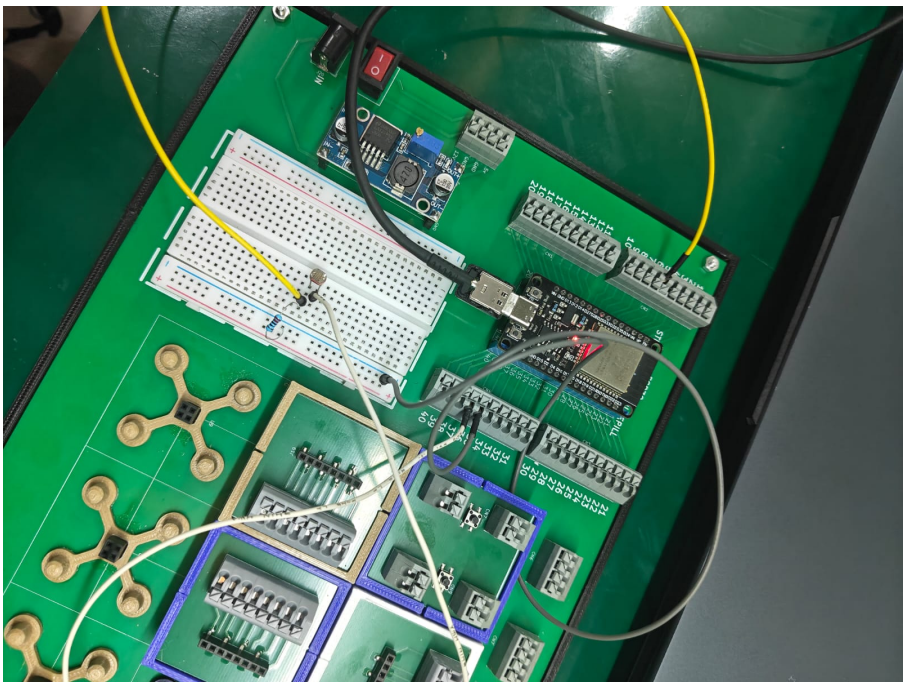
ubah halus, sehingga ADC dapat merepresentasikan banyak tingkat terang dan gelap, bukan hanya dua kondisi.

4 Kesimpulan

Berdasarkan praktikum yang sudah kami lakukan, program Blink berhasil di running pada STM32 Blackpill dengan framework Arduino melalui ST-Link dan pada ESP32 dengan framework ESP-IDF, meskipun proses *build* sempat mengalami kendala yang bersumber dari lingkungan pengembangan dan bukan dari kode. Konfigurasi *pull-up* dan *pull-down*, baik eksternal maupun internal, terbukti memberikan level input yang pasti pada pin GPIO dengan logika pembacaan yang saling berkebalikan. Debouncing diperlukan untuk menghilangkan efek getaran tombol agar satu tekanan hanya terhitung satu kali, dengan nilai delay yang dipilih sebagai kompromi antara kebersihan sinyal dan kecepatan respons. Selain itu, ADC memungkinkan GPIO membaca sinyal analog dengan 4096 level, sehingga perubahan intensitas cahaya pada LDR dapat terbaca sebagai nilai yang kontinu.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 19: Perangkaian percobaann menggunakan LDR